

Linguagens Regulares vs. Linguagens Livres de Contexto (LLC)

A principal diferença entre esses dois tipos de linguagens está no **poder de expressão** e na capacidade de processar estruturas complexas.

1. Linguagens Regulares (Mais Simples)

- **O que são:** São altamente eficientes e fáceis de implementar, mas possuem regras rígidas e limitações. Elas não têm "memória" para contar ou emparelhar elementos distantes.
- **Exemplo prático:** Expressões Regulares (Regex) usadas para validar formatos de texto.
 - *Cenário:* Validar se um e-mail possui @ e .com , ou se um número de telefone está no formato (XX) XXXXX-XXXX .
 - *Padrão:* Uma sequência simples como "um número seguido de qualquer quantidade de letras" (1a, 1aaaa, 1abc).

2. Linguagens Livres de Contexto (Mais Complexas)

- **O que são:** São muito mais amplas. Elas englobam tudo o que as linguagens regulares fazem, mas conseguem representar estruturas hierárquicas e aninhadas através de **árvores de derivação** (estruturas em formato de ramificações).
- **Exemplo prático:** A sintaxe de linguagens de programação (como Python, C ou Java) e expressões matemáticas.
 - *Cenário:* Balanceamento de parênteses ou chaves. Para cada chave que se abre { , o sistema precisa saber que uma chave deve se fechar } mais adiante, mesmo que haja muito código lá dentro:

JSON



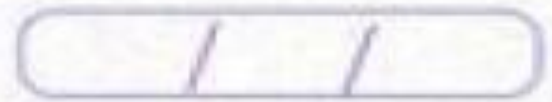
```
{  
  "usuario": {  
    "id": 1,  
    "ativo": true  
  }  
}
```

Java

```
boolean logado = true;
boolean admin = true;

if (logado) {
    if (admin) {
        System.out.println("Acesso liberado!");
    }
}
```

2



$$(23+3)-(34-(34 \times 2)) \times (-4) = ?$$

$$(12 + 12) \times (23-3)+(32 \times (23-2) \times 3) = ?$$

$$23 \div 20 + 23 \times 2 - 23 = ?$$

Resumo Comparativo

Característica	Linguagem Regular	Linguagem Livre de Contexto (LLC)
Complexidade	Baixa (regras rígidas e lineares)	Alta (estruturas profundas e ramificadas)
Memória	Não tem capacidade de contagem	Consegue "lembrar" de elementos aninhados
Onde é usada?	Validação de campos (Regex, busca de texto)	Compiladores (análise da sintaxe de códigos)

A análise sintática checa se a **combinação e a ordem** dessas palavras fazem sentido.

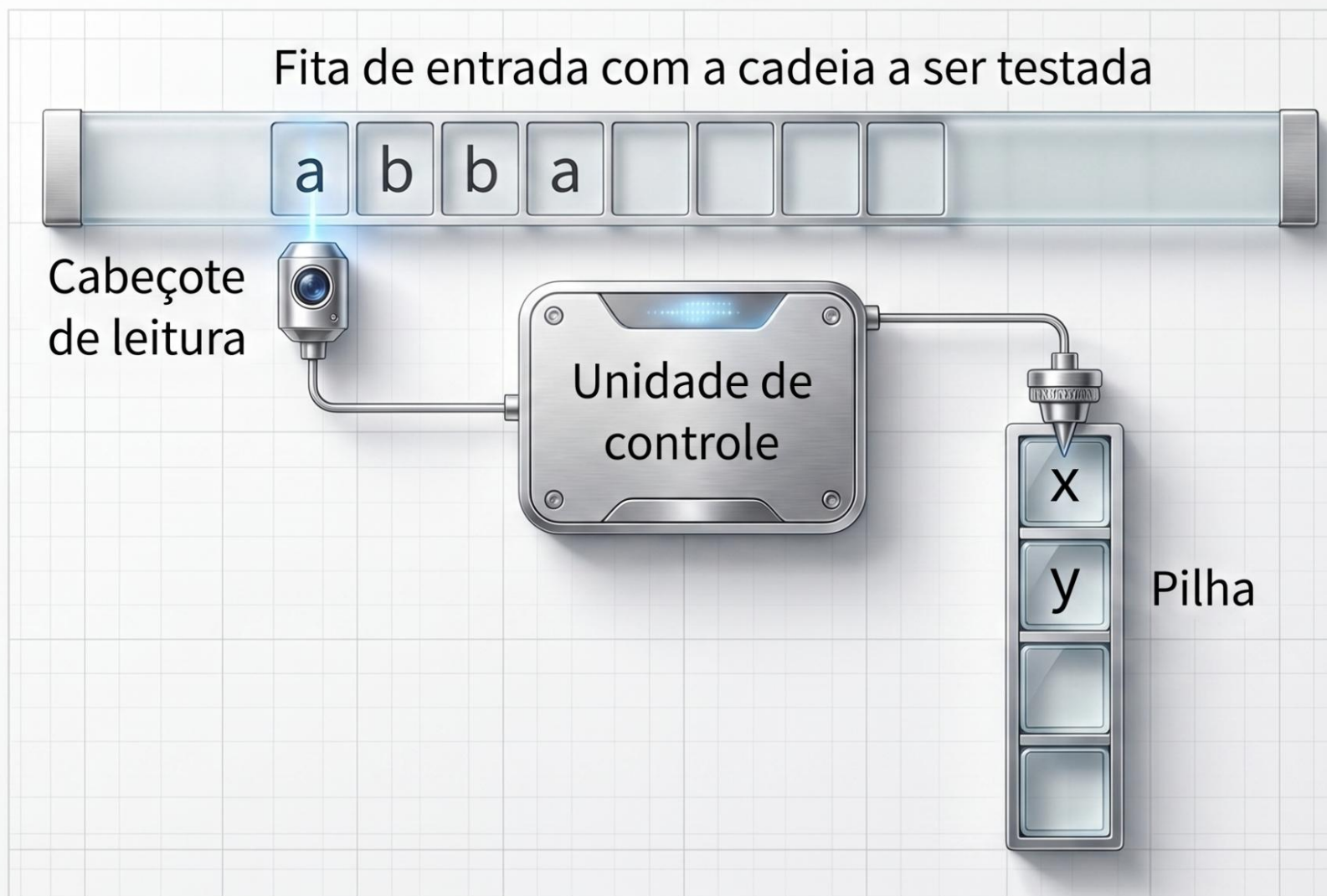
5.5. Autômato de Pilha (AP)

Como as **Linguagens Livres de Contexto** são mais complexas que as regulares, os autômatos finitos comuns (como o AFND) não conseguem reconhecê-las porque não têm memória.

Para resolver isso, usamos o **Autômato de Pilha**. Ele possui a mesma estrutura de um AFND, mas ganha um recurso extra: uma **memória auxiliar em forma de pilha**.

- **Como funciona essa memória?** Ela segue a lógica **LIFO** (*Last In, First Out*): o último símbolo empilhado é, obrigatoriamente, o primeiro a ser retirado (desempilhado).
- **Para que serve?** Essa pilha permite ao autômato "lembrar" o que já leu para conseguir casar elementos mais adiante (como garantir que cada parêntese aberto tenha seu fechamento correspondente).

Figura 4: Representação de um autômato com pilha



Para definir esse autômato, usamos uma sêxtupla (6 elementos):

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- **Q : Conjunto de estados** (as fases ou momentos da máquina).
- (Sigma) • **Σ : Alfabeto de entrada** (os símbolos que a máquina lê do texto).
- (Gama) • **Γ : Alfabeto da pilha** (os símbolos que a máquina pode guardar na memória). *Esta é a única novidade em relação ao AFD.*
- (Delta) • **δ : Função de transição** (as regras que dizem: "se eu estiver no estado X , ler o símbolo Y e o topo da pilha for Z , vou para o estado W ").
- **q_0 : Estado inicial** (onde o processo começa).
- **F : Estados finais** (as fases de sucesso; se a palavra terminar aqui, ela é aceita).

O que você precisa saber sobre o Autômato a Pilha (AP)

- **O que ele reconhece:** Assim como os autômatos finitos (AFD) lidam com gramáticas regulares, os autômatos a pilha são projetados para reconhecer **Gramáticas Livres de Contexto (GLC)**.
- **A Memória Auxiliar:** O grande diferencial é o acesso a uma memória em formato de **pilha** que é independente da fita de entrada e, teoricamente, **infinitamente longa**.

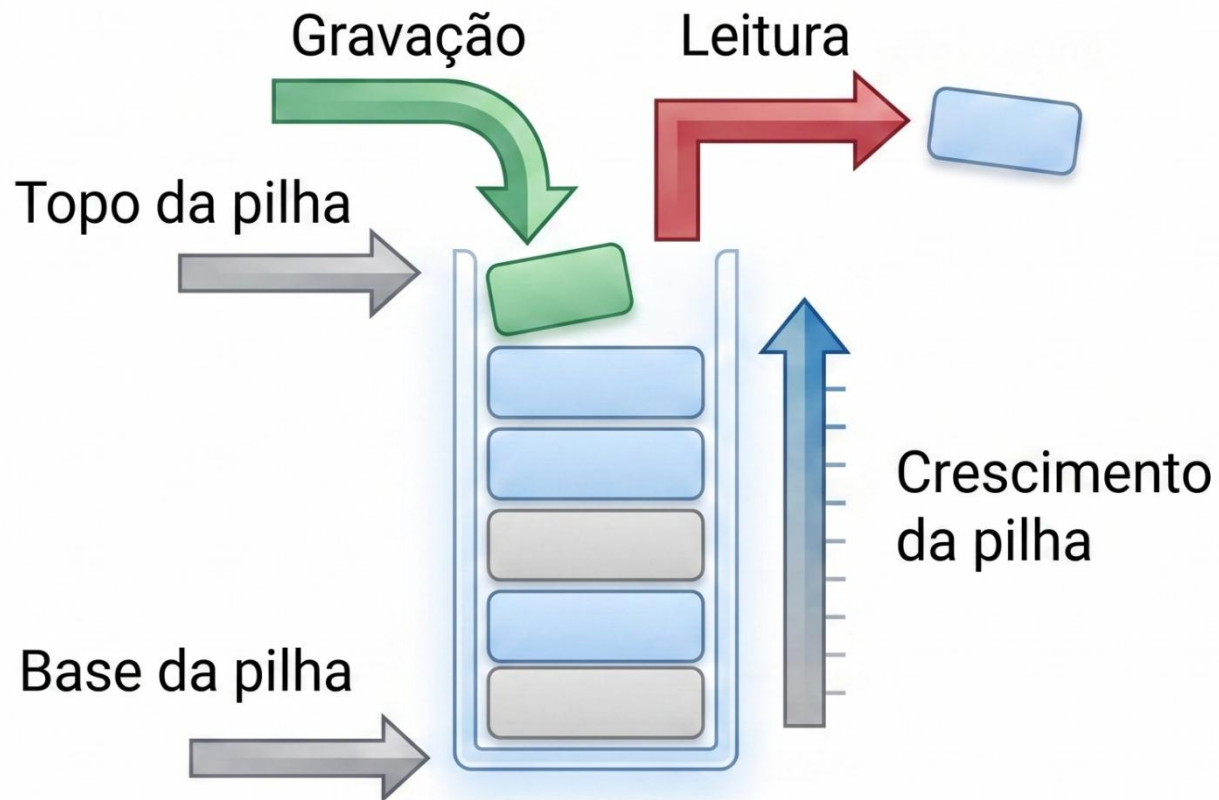
Por que esse nome?

O termo "**Livre de Contexto**" significa que as regras de substituição são aplicadas de forma totalmente isolada, **sem depender dos símbolos que estão ao redor**.

- Se uma regra diz que o símbolo **A** pode ser substituído por **B**, essa troca será feita sempre, não importa se **A** está sozinho, no começo de uma linha ou cercado por outros caracteres.
- O "contexto" vizinho não altera e nem impede a aplicação da regra.

- **A Lógica LIFO (*Last In, First Out*):** O funcionamento segue a estrutura clássica de empilhamento (como uma pilha de pratos):
 - A **base é fixa** e o **topo é dinâmico** (muda conforme o volume de dados).
 - O **último** símbolo gravado é, obrigatoriamente, o **primeiro** a ser recuperado.
- **Dinâmica de Execução:** A cada caractere lido na entrada, o autômato consulta suas regras de transição para decidir se vai **empilhar (push)** um novo símbolo, **desempilhar (pop)** o que está no topo ou apenas avançar sem alterar a memória.

Figura 5: Representação de um autômato com pilha



1. Critérios de Aceitação (Como a máquina valida a palavra)

Para processar uma cadeia, o autômato começa no estado inicial q_0 com o símbolo base na pilha. Ao terminar de ler o texto, a aceitação pode acontecer de duas formas (dependendo de como o autômato foi projetado):

- **Por Estado Final:** A leitura acaba e o autômato está em um estado de aceitação (F). O que sobrou na pilha não importa.
- **Por Pilha Vazia:** A leitura acaba e a pilha foi totalmente esvaziada.

2. A Definição Formal (Agora com 7 elementos)

Para incluir o controle do fundo da pilha, adicionamos o símbolo Z_0 à sêxtupla, transformando-a em uma sêxtupla/sétupla:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

- Z_0 (ou \$): É o **símbolo inicial da pilha**. Ele serve como o "chão" da memória. Se ele for o único elemento visível, você sabe que a pilha está vazia.

Configuração Inicial do Sistema: É representada por (q_0, w, Z_0) , significando:

Estado inicial (q_0), palavra a ser lida (w) e pilha contendo apenas a base (Z_0).

3. A Dinâmica de Movimento (Função de Transição δ)

A cada passo da execução, a máquina analisa o cenário atual para decidir o próximo movimento. A regra segue este formato:

$$\delta(q, a, Z) = \langle t, \gamma \rangle$$

O que a máquina olha (Entrada):

- q : Estado atual em que a máquina se encontra.
- a : Caractere sendo lido na fita de entrada.
- Z : Símbolo que está atualmente no topo da pilha (e que será consumido/removido).

O que a máquina faz (Saída):

- t : O próximo estado para onde a máquina vai.

Gama • γ : O novo símbolo (ou conjunto de símbolos) que será colocado no topo da pilha para substituir o Z .

execução prática de um **Autômato com Pilha (AP)** usando o critério de **Aceitação por Pilha Vazia** ($F = \text{vazio}$ ou ϵ). Isso significa que a palavra só é aceita se, ao terminar de ler a entrada, a pilha for completamente esvaziada.

Exemplo 1: Mesma quantidade de 0s e 1s (Em qualquer ordem)

Objetivo: Aceitar cadeias como 0101, 1100, 0011. A ordem não importa, contanto que o número final de zeros e uns seja idêntico.

A Lógica do "Cancelamento"

- Se o símbolo lido for **igual** ao que está no topo da pilha (ou se a pilha estiver só com a base Z_0), você **empilha**.
- Se o símbolo lido for **diferente** (um 0 lido contra um 1 no topo, ou vice-versa), eles se anulam: você **desempilha**.

Notação de Definição de Conjunto

$$L = \{w \in \{0, 1\}^* \mid n_0(w) = n_1(w)\}$$

"O conjunto L ..."	"...é igual a..."	"...o conjunto de..."	"...toda cadeia w pertencente ao alfabeto $\{0, 1\}$ fecho de Kleene..."
"...tal que..."	"...o número de 0s em w é igual ao número de 1s em w ."		(Fim da definição)

Regra de Transição $\delta(q, a, Z) = \langle t, \gamma \rangle$

Estado q Caractere Lido a Símbolo no Topo da Pilha Z Próximo Estado t Símbolo colocado Topo da Pilha γ

Gama

1. Empilhando o excesso (quando a pilha está "neutra" ou com o mesmo símbolo):

- $\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$
- $\delta(q_0, 1, Z_0) = (q_0, 1Z_0)$
- $\delta(q_0, 0, 0) = (q_0, 00)$
- $\delta(q_0, 1, 1) = (q_0, 11)$

Regra de Transição $\delta(q, a, Z) = \langle t, \gamma \rangle$

Estado δ Caractere Lido a Símbolo no Topo da Pilha Z

Gama $\langle t, \gamma \rangle$ Próximo Estado t Símbolo colocado Topo da Pilha γ

2. Anulando símbolos (quando o símbolo lido é oposto ao do topo):

- $\delta(q_0, 0, 1) = (q_0, \epsilon)$
- $\delta(q_0, 1, 0) = (q_0, \epsilon)$

Regra de Transição $\delta(q, a, Z) = \langle t, \gamma \rangle$

Estado q Caractere Lido a Símbolo no Topo da Pilha Z Próximo Estado t Símbolo colocado Topo da Pilha γ

Gama

3. Finalização (quando a cadeia termina e a pilha volta ao estado inicial):

- $\delta(q_0, \epsilon, Z_0) = (q_f, Z_0)$

Funções de Transição (δ)

$$L = \{w \in \{0, 1\}^* \mid n_0(w) = n_1(w)\}$$

- $\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$
- $\delta(q_0, 1, Z_0) = (q_0, 1Z_0)$
- $\delta(q_0, 0, 0) = (q_0, 00)$
- $\delta(q_0, 1, 1) = (q_0, 11)$
- $\delta(q_0, 0, 1) = (q_0, \epsilon)$
- $\delta(q_0, 1, 0) = (q_0, \epsilon)$
- $\delta(q_0, \epsilon, Z_0) = (q_f, Z_0)$

Rastreamento da Cadeia 0110			
Passo	Estado	Entrada Restante	Pilha (Topo à esquerda)
0	q_0	0110	Z_0
1	q_0	110	$0Z_0$
2	q_0	10	Z_0
3	q_0	0	$1Z_0$
4	q_0	ϵ	Z_0
5	q_f	ϵ	Z_0

No diagrama de um AP, as transições são representadas no formato $a, b \rightarrow c$, onde:

- a é o símbolo lido da fita de entrada.
- b é o símbolo desempilhado (topo da pilha).
- c é o símbolo (ou símbolos) empilhado(s).

$0, Z_0 \rightarrow 0Z_0$

$1, Z_0 \rightarrow 1Z_0$

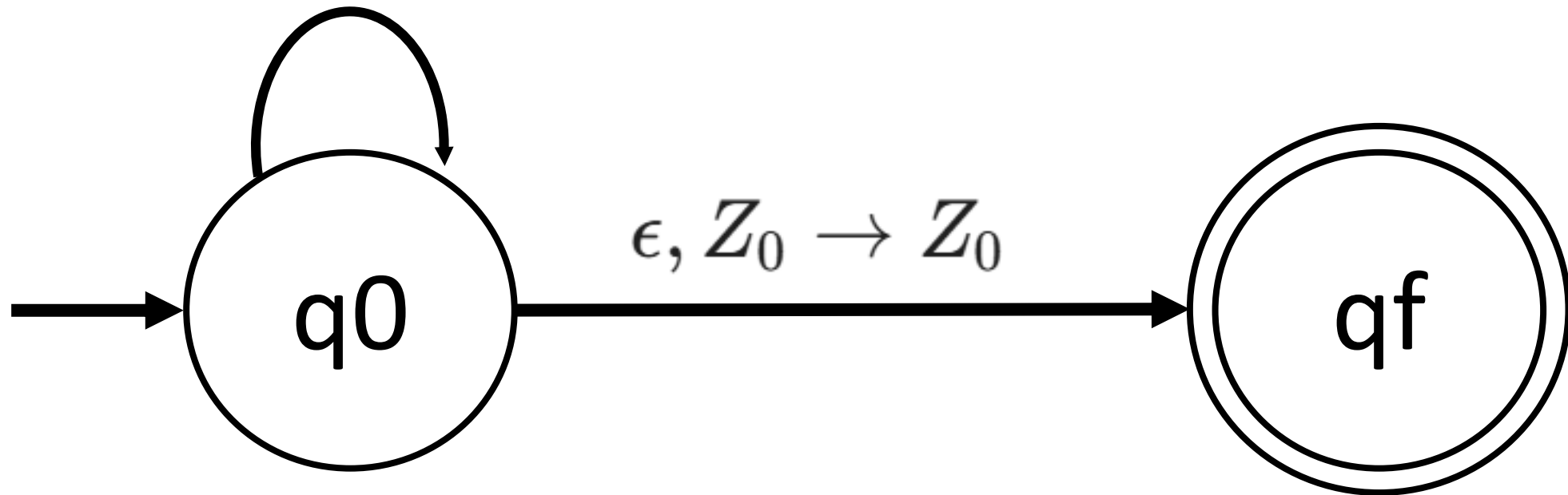
$0, 0 \rightarrow 00$

$1, 1 \rightarrow 11$

$0, 1 \rightarrow \epsilon$

$1, 0 \rightarrow \epsilon$

símbolo lido $\rightarrow a, b \rightarrow c$
desempilhado (topo da pilha) $\uparrow$ símbolo (ou símbolos) empilhado(s)



Realize o rastreamento (trace) da cadeia de entrada
 $w = 11100010$

Tabela de Rastreamento

Passo	Estado	Entrada Restante	Pilha (Topo à esquerda)
-------	--------	------------------	-------------------------

Passo	Estado	Entrada Restante	Pilha (Topo à esquerda)
0	q_0	11100010	Z_0
1	q_0	1100010	$1Z_0$
2	q_0	100010	$11Z_0$
3	q_0	00010	$111Z_0$
4	q_0	0010	$11Z_0$
5	q_0	010	$1Z_0$
6	q_0	10	Z_0
7	q_0	0	$1Z_0$
8	q_0	ϵ	Z_0
9	q_f	ϵ	Z_0

Exemplo 2: A Linguagem $0^n 1^n$ (Ordem Exata)

Objetivo: Aceitar cadeias onde todos os 0s vêm primeiro, seguidos pela **mesma** quantidade de 1s (ex: 0011, 000111).

A Lógica dos Estados

Aqui precisamos de **dois estados** para garantir a ordem das letras:

- **Estado q_0 :** Fase de acumular os 0s na pilha.
- **Estado q_1 :** Fase de ler os 1s e ir eliminando os 0s da pilha.

Notação de Definição de Conjunto

$$L = \{0^n 1^n \mid n \geq 0\}$$

Componentes da Definição

- 0^n : Representa uma sequência de n zeros consecutivos.
- 1^n : Representa uma sequência de n uns consecutivos.
- $n \geq 0$: Especifica que a quantidade n deve ser a mesma para ambos os símbolos e pode ser qualquer número inteiro não negativo.
- **Cadeia Vazia (ϵ ou λ):** Como o intervalo de n inclui o zero, a cadeia vazia pertence a esta linguagem ($0^0 1^0 = \epsilon$).

A Função de Transição (δ)

1. $\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$: Ao ler o primeiro 0, ele é colocado na pilha sobre o marcador de fundo.
2. $\delta(q_0, 0, 0) = (q_0, 00)$: Enquanto ler zeros, continue empilhando-os em q_0 .

A Função de Transição (δ)

3. $\delta(q_0, 1, 0) = (q_1, \epsilon)$: Ao ler o primeiro 1, um 0 é removido e o autômato muda para o estado q_1 (fase de conferência).
4. $\delta(q_1, 1, 0) = (q_1, \epsilon)$: No estado q_1 , para cada 1 lido, remove-se um 0.

A Função de Transição (δ)

5. $\delta(q_0, \epsilon, Z_0) = (q_2, Z_0)$: Se a entrada for vazia ($n = 0$), o autômato aceita imediatamente.
6. $\delta(q_1, \epsilon, Z_0) = (q_2, Z_0)$: Se a fita acabou e a pilha voltou ao marcador inicial, a cadeia é aceita.

A Função de Transição (δ)

$$1. \delta(q_0, 0, Z_0) = (q_0, 0Z_0)$$

$$2. \delta(q_0, 0, 0) = (q_0, 00)$$

$$3. \delta(q_0, 1, 0) = (q_1, \epsilon)$$

$$4. \delta(q_1, 1, 0) = (q_1, \epsilon)$$

$$5. \delta(q_0, \epsilon, Z_0) = (q_2, Z_0)$$

$$6. \delta(q_1, \epsilon, Z_0) = (q_2, Z_0)$$

$$L = \{0^n 1^n \mid n \geq 0\}$$

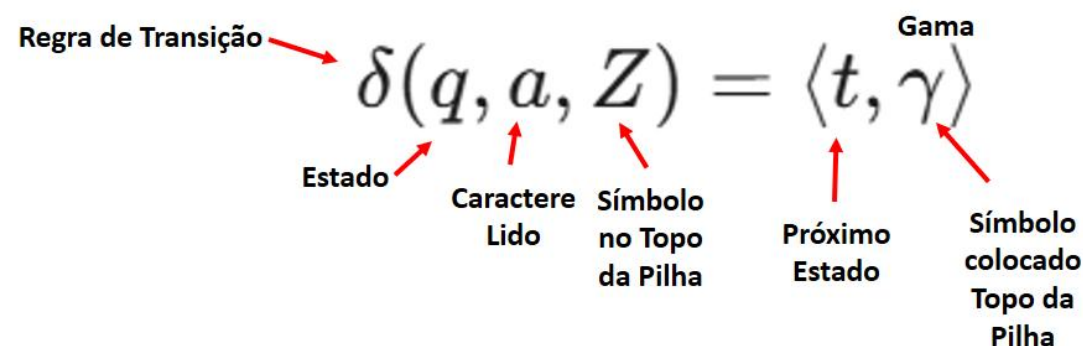


Tabela de Rastreamento (Trace) para 0011

Passo	Estado	Entrada Restante	Pilha (Topo à esquerda)
-------	--------	------------------	-------------------------

Tabela de Rastreamento (Trace) para 0011

Passo	Estado	Entrada Restante	Pilha (Topo à esquerda)
0	q_0	0011	Z_0
1	q_0	011	$0Z_0$
2	q_0	11	$00Z_0$
3	q_1	1	$0Z_0$
4	q_1	ϵ	Z_0
5	q_2	ϵ	Z_0

Faça o Autômato de Pilha para essa linguagem

1. $\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$
2. $\delta(q_0, 0, 0) = (q_0, 00)$
3. $\delta(q_0, 1, 0) = (q_1, \epsilon)$
4. $\delta(q_1, 1, 0) = (q_1, \epsilon)$
5. $\delta(q_0, \epsilon, Z_0) = (q_2, Z_0)$
6. $\delta(q_1, \epsilon, Z_0) = (q_2, Z_0)$

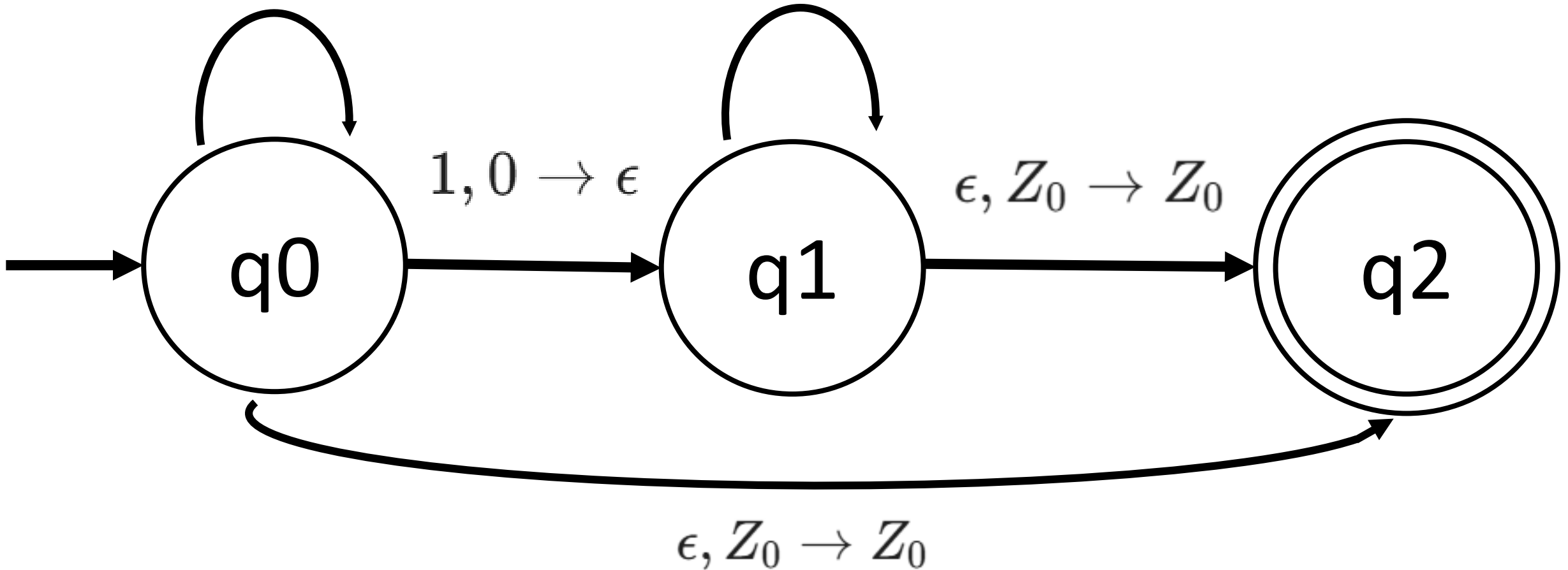
símbolo lido $\rightarrow a, b \rightarrow c$
desempilhado (topo da pilha) $\uparrow$ símbolo (ou símbolos) empilhado(s)

$0, Z_0 \rightarrow 0Z_0$

$0, 0 \rightarrow 00$

símbolo lido $\rightarrow a, b \rightarrow c$
desempilhado (topo da pilha) $\uparrow$ símbolo (ou símbolos) empilhado(s)

$1, 0 \rightarrow \epsilon$



Exercício: O Sinal com Eco ($0^n 1^{n+1}$)

$$L = \{0^n 1^{n+1} \mid n \geq 0\}$$

Realize o rastreamento da cadeia $w = 011$.

Funções de Transição (δ):

1. $\delta(q_0, 0, Z_0) = (q_0, 0Z_0)$ — (Primeiro 0)
2. $\delta(q_0, 0, 0) = (q_0, 00)$ — (Acumula 0s)
3. $\delta(q_0, 1, Z_0) = (q_f, Z_0)$ — (Caso $n = 0$: apenas um '1' é lido)
4. $\delta(q_0, 1, 0) = (q_1, \epsilon)$ — (Primeiro 1 cancela um 0)
5. $\delta(q_1, 1, 0) = (q_1, \epsilon)$ — (Outros 1s cancelam 0s)
6. $\delta(q_1, 1, Z_0) = (q_f, Z_0)$ — (O último '1' extra leva à aceitação)

Conclusões do Capítulo

- **Limitação dos Autômatos Finitos:** Embora as linguagens regulares sejam muito úteis, a falta de memória dos autômatos finitos (AF) limita sua aplicação a regras e gramáticas muito simples.
- **O Diferencial da Pilha:** Ao adicionar uma **memória em pilha**, expandimos a capacidade computacional para as **Gramáticas Livres de Contexto (GLC)**, permitindo gerar e validar cadeias de texto bem mais complexas.
- **Hierarquia de Chomsky:** O avanço nessa estrutura funciona como uma escada de poder: quanto mais alto o nível na hierarquia, mais expressiva é a gramática e mais poderoso (e complexo) precisa ser o motor que a reconhece.